Knowledge-Guided Requirement Feasibility Analysis via Multi-Agent Reasoning

Anonymous Author(s)

Abstract

Natural language requirements often contain unrealistic goals, infeasible constraints, and implicit assumptions about resources, performance, and deployment environments. This paper studies requirement feasibility as a first-class requirements engineering problem. We propose a research framework for knowledge-guided feasibility analysis in which multiple specialized agents inspect candidate requirements, reason over constraint categories, and surface unrealistic or underspecified assumptions for human validation. The paper is structured around requirement realism verification rather than system construction: it defines the target problem, outlines a multi-agent verification workflow, and specifies an evaluation plan for measuring feasibility detection, constraint verification, and human review reduction.

Keywords

requirements engineering, requirement feasibility, requirement validation, constraint reasoning, multi-agent reasoning, human-in-the-loop

1 Introduction

1.1 Background and Motivation

Requirements determine what a software project is expected to deliver, but natural language requirements frequently include goals that are unrealistic under available resources, deployment environments, performance budgets, or organizational constraints. Recent work has highlighted the central role of requirements in AI-driven software engineering [2] and the need to reduce ambiguity while preserving stakeholder readability in requirement specification [1]. This paper focuses on *requirement feasibility*: whether a stated requirement is realistic, implementable, and deployable under explicit or implicit constraints.

1.2 Challenges of Requirement Feasibility Analysis

Feasibility analysis is difficult because many infeasible requirements are not syntactically invalid. They may look precise while hiding resource limits, temporal assumptions, hardware dependencies, data availability assumptions, or deployment restrictions. LLM-based requirement generation can amplify this issue by producing fluent but unrealistic requirements whose feasibility has not been checked against project knowledge.

1.3 Multi-Agent Requirement Verification

We study a lightweight multi-agent verification workflow in which agents play complementary analysis roles: requirement interpretation, constraint verification, and resource/deployment feasibility checking. The goal is not to build a large platform, but to decompose feasibility verification into focused reasoning steps that can be inspected and corrected by humans.

1.4 Contributions

This paper makes the following planned contributions:

- A problem formulation for knowledge-guided requirement feasibility analysis and requirement realism verification.
- A multi-agent workflow for identifying unrealistic requirements, infeasible constraints, and missing feasibility assumptions.
- A constraint reasoning scheme covering resource, performance, deployment, and temporal feasibility.
- An experimental design for evaluating feasibility detection accuracy, constraint verification accuracy, unrealistic requirement detection, and human verification reduction.

2 Related Work

2.1 Requirement Validation

Requirement validation checks whether requirements are correct, complete, consistent, and acceptable to stakeholders. This paper narrows the validation scope to feasibility and realism: whether a requirement can plausibly be implemented and deployed under known constraints.

2.2 Requirement Feasibility Analysis

Feasibility analysis studies whether requirements can be satisfied given technical, organizational, economic, and operational limits. Our work treats feasibility as an explicit verification target rather than as an informal review activity.

2.3 Constraint Analysis in Requirements Engineering

Constraint analysis provides a basis for identifying limits related to resources, performance, deployment, and time. The proposed study uses constraint categories as the organizing structure for feasibility reasoning.

2.4 LLM-Based Requirements Engineering

LLMs can help generate, rewrite, and inspect requirements, but generated requirements may be fluent without being realistic. We position LLM-based agents as feasibility reviewers whose outputs must be grounded in knowledge and validated by humans.

2.5 Knowledge-Guided Requirement Analysis

Knowledge-guided analysis incorporates domain knowledge, requirement patterns, constraint rules, and project context into requirement inspection. This direction is closely related to controlled and structured requirement specification efforts [1].

2.6 Multi-Agent Collaboration for Requirement Verification

Multi-agent collaboration can separate interpretation, constraint checking, and feasibility judgment into specialized roles. In this paper, agent collaboration is used as a practical workflow for requirement verification, not as a general-purpose software development framework.

3 Methodology

3.1 Framework Overview

The proposed framework verifies the feasibility of natural language requirements before downstream development activities. Given a requirement statement and available project knowledge, the workflow produces a feasibility judgment, supporting constraint evidence, detected realism issues, and suggested requirement refinements.

Scope. This study focuses only on G6-feasibility: requirement feasibility analysis and realism verification. It does not address requirement graph construction, conflict detection, architecture mapping, executable requirements, requirement planning, or dynamic requirement clarification.

3.2 Multi-Agent Feasibility Analysis Workflow

3.2.1 Analyst Agent. The analyst agent normalizes the input requirement, identifies the intended capability, and extracts feasibility-relevant assumptions. Its output is a structured requirement interpretation used by the downstream verification agents.

3.2.2 Constraint Verification Agent. The constraint verification agent checks whether the requirement contains explicit or implicit constraints that may exceed project limits. It focuses on constraint categories rather than broad requirement quality issues.

3.2.3 Deployment and Resource Agent. The deployment and resource agent inspects operational assumptions such as infrastructure availability, runtime environment, data access, hardware limits, and staffing or budget constraints. It reports whether the requirement appears deployable under the available context.

3.3 Requirement Feasibility Detection

3.3.1 Resource Constraints. Resource constraints include budget, staffing, hardware capacity, data availability, and external service dependencies.

3.3.2 Performance Constraints. Performance constraints include latency, throughput, scalability, reliability, and accuracy targets that may be unrealistic for the stated system context.

3.3.3 Deployment Constraints. Deployment constraints include platform compatibility, network availability, privacy restrictions, operating environments, and integration conditions.

3.3.4 Temporal Constraints. Temporal constraints include deadlines, real-time behavior, release schedules, and maintenance windows that may conflict with implementation feasibility.

3.4 Knowledge-Guided Constraint Analysis

3.4.1 Requirement Knowledge. Requirement knowledge includes domain terms, stakeholder assumptions, requirement templates, and known feasibility risks.

3.4.2 Constraint Rules. Constraint rules encode feasibility checks such as resource budget limits, deployment prerequisites, and incompatible constraint combinations.

3.4.3 Domain Knowledge. Domain knowledge provides contextual evidence for judging whether a requirement is realistic in a target application domain.

3.4.4 Prompt Strategies. Prompt strategies guide agents to provide structured judgments, constraint evidence, uncertainty labels, and refinement suggestions.

3.5 Requirement Verification and Refinement

3.5.1 Feasibility Verification. The final feasibility decision aggregates agent findings into labels such as feasible, conditionally feasible, infeasible, or insufficient information.

3.5.2 Unrealistic Requirement Detection. Unrealistic requirements are flagged when stated goals conflict with known constraints, require unavailable resources, or assume unsupported deployment conditions.

3.5.3 Requirement Adjustment Suggestions. For infeasible or conditionally feasible requirements, the workflow suggests conservative refinements such as relaxing targets, adding missing assumptions, or replacing unverifiable claims with measurable constraints.

3.6 Human-in-the-Loop Interaction

Human reviewers inspect the agents' feasibility judgments, confirm domain assumptions, and decide whether suggested refinements should be accepted. The human-in-the-loop step is designed to reduce review effort while preserving stakeholder authority over requirement decisions.

4 Experiments

4.1 Research Questions

The evaluation is organized around the following research questions:

- **RQ1:** How accurately does the workflow detect infeasible or conditionally feasible requirements?
- **RQ2:** How accurately does it identify the underlying constraint category?
- **RQ3:** How much do knowledge guidance and multi-agent role separation contribute to the results?
- **RQ4:** How much human verification effort can be reduced without removing human decision authority?

4.2 Experimental Setup

The experimental setup will define the input requirement format, available project knowledge, agent prompts, constraint rule sources, and human annotation protocol. All implementation details should be reported sufficiently for reproduction.

4.3 Datasets

The dataset should contain natural language requirements annotated with feasibility labels and constraint categories. Candidate sources include course project requirements, public requirements engineering artifacts, and manually constructed infeasible variants. The final paper should report dataset size, domains, annotation guidelines, and inter-annotator agreement.

4.4 Baselines

Baselines should compare the proposed workflow against simpler feasibility review strategies:

- Single-agent requirement feasibility review.
- Rule-only constraint checker.
- Multi-agent review without knowledge guidance.
- Human-only review used as the reference process.

4.5 Evaluation Metrics

4.5.1 Feasibility Detection Accuracy. Measures whether the workflow correctly classifies requirements as feasible, conditionally feasible, infeasible, or insufficient information.

4.5.2 Constraint Verification Accuracy. Measures whether the identified constraint category matches the human-annotated reason for feasibility risk.

4.5.3 Unrealistic Requirement Detection. Measures precision, recall, and F1 for requirements judged unrealistic under the available project context.

4.5.4 Human Verification Reduction. Measures the reduction in manual review effort, such as fewer requirements requiring full expert inspection or fewer clarification rounds.

4.6 Main Results

This subsection will report the main quantitative comparison after the dataset and baselines are finalized. No experimental results are claimed at the structure-design stage.

4.7 Ablation Study

The ablation study should isolate the effect of role separation, knowledge guidance, constraint rules, and human feedback. The expected output is an evidence-based explanation of which components improve feasibility verification.

4.8 Case Study

The case study should present a small number of representative requirements, agent findings, human judgments, and final refinements. It should illustrate the feasibility analysis workflow without overgeneralizing from anecdotal examples.

5 Threats to Validity

5.1 Internal Validity

Internal validity threats include annotation bias, inconsistent human feasibility judgments, prompt sensitivity, and possible leakage between knowledge sources and evaluation examples. The study should mitigate these risks through clear annotation guidelines, independent review, and prompt/version reporting.

5.2 External Validity

External validity threats concern whether results generalize across domains, requirement styles, project sizes, and deployment environments. The evaluation should therefore report domain coverage and avoid claiming generality beyond the evaluated requirement corpus.

5.3 Construct Validity

Construct validity threats arise if feasibility, realism, or human review reduction are measured too narrowly. The metric design should align with the research questions and distinguish feasibility problems from broader requirement quality issues.

6 Conclusion

This paper frames requirement feasibility analysis as a research problem in intelligent requirements engineering. The proposed structure centers on knowledge-guided constraint reasoning, multi-agent requirement verification, and human-in-the-loop validation. Rather than presenting a broad platform, the paper focuses on whether natural language requirements are realistic, implementable, and deployable under explicit or implicit project constraints.

References

[1] Ikram Darif, Ghizlane El Boussaidi, Segla Kpodjedo, and Cristiano Politowski. 2026. Controlled Natural Language for Requirements Specification: A Systematic Literature Review. *ACM Comput. Surv.* 58, 7 (Jan. 2026), 175:1–175:36. doi:10.1145/3778169

[2] Diana Robinson, Christian Cabrera, Andrew D. Gordon, Neil D. Lawrence, and Lars Mennen. 2025. Requirements Are All You Need: The Final Frontier for End-User Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (May 2025), 141:1–141:22. doi:10.1145/3708524

A Appendix

The appendix may include the annotation guideline, feasibility label definitions, prompt templates, constraint rule examples, and additional case study material. It should not introduce new research questions beyond requirement feasibility analysis.